

Assignment 3

Instructions

- PHY410: Do problems 1, 2a, and 3a-b
- PHY 505: Do all problems

Accept the assignment from “Classroom 50” website: <https://classroom50.org/ubsuny/compphys-fall26/assignments/assignment-3/accept>. This will create a new repository for you on github, titled something like `github.com/ubsuny/ubsuny/compphys-fall26-assignment-3-username`. The repository is located in the `ubsuny` github group, but it is your personal repository, and only you can view it. (The repository is actually not a fork, but rather a brand new repository created by directly copying files from a “template repository.”)

From here on out, the assignments will largely use Jupyter notebooks. You should still upload your code to GitHub and a “lab report”/“write-up”-style document to UBLearn. However, for the write-up, I suggest that you use Markdown cells in the notebooks themselves and export the notebook to PDF/HTML/etc., rather than preparing a separate document (this might not always be possible, e.g., if you want to scan a handwritten document, feel free to do so). Like previous assignments, you can upload as many times as you like before the due date.

You can use AI to help write code (it is not necessary, though). Any interpretations and explanations must be your own. Describe any use of AI in your answer.

Problem 1: CO2 fitting

25 points

Please do your work in `problem1/problem1.ipynb`, which is provided in this repo.

Problem 1a

5 points

The Mauna Loa CO2 data set provides a nice example for Fourier transforms, but, as you well know, its primary interest lies in the overall rise, not the annual oscillations. First, download the *yearly mean* CO2 data from <https://gml.noaa.gov/ccgg/trends/data.html>. Load it into a numpy array and make a matplotlib plot with year on the x-axis and mean CO2 on the y-axis. Include error bars on the plot.

Problem 1b

10 points

Let's try to model the change since the industrial revolution. Subtract off 280ppm from your data (the approximate value before the industrial revolution), and then fit the offset data with an exponential function. Recall that you can perform an exponential fit with the linear `chi_square_fit()` function with an appropriate transformation of the data.

1. Print the fit parameters, uncertainties, and quality of fit. Note: the quality of fit might not be very good! Comment on this in your writeup.
2. Plot the fit on top of the data (remember to account for the offset of 280 ppm before plotting).
3. At 50,000 ppm CO2 concentration, the atmosphere will become toxic to oxygen-breathing life. Based on your exponential fit, in what year will the atmosphere become toxic? (Note: this is not a realistic estimate of anything, the world will run out of fossil fuels before this happens.)

Problem 1c

10 points

Repeat 1b, except fit with a quadratic polynomial using `np.polyfit`. Compare and comment on the predictions for 50,000 ppm.

Note: to improve the stability of the fit, you can center the x-coordinates, e.g., take `year-2000` instead of just `year`. Remember to account for this when you plot the results.

Problem 2: CO2 Fourier analysis

25 or 35 points

PHY410 students: do 2a only

PHY505 students: do full problem

Please do the problem in Jupyter in `problem2/problem2.ipynb`.

In this problem, we will repeat some of the in-class activities with the sunspot data set on the *monthly* CO2 dataset. The dataset has already been downloaded to `problem2/co2_mm_mlo.txt`. Using some combination of waveform modification (which may include subtracting the offset, padding, windowing, taking the FFT, manipulating the waveforms, inverse FFT, and undoing the window+padding), do the following:

Problem 2a

25 points

Plot the default and smoothed Fourier power spectrum of the full CO2 data. You should zero-pad the input array to the nearest power of 2. Choose your favorite window function.

For both spectra, make two plots:

1. Power spectrum versus the Fourier component, k
2. Power spectrum versus the period in months (this should reveal the annual modulation much more obviously)

Problem 2b

10 points

Clean up the jitters (high frequency noise, index > 100) in the time domain by zeroing the appropriate waveform coefficients in the frequency domain and doing the inverse Fourier transform. Plot both the “raw” and “cleaned” time series in the time domain. (Note: this is for demonstration and educational purposes; it is not necessarily what you would do in real life.) Show a zoomed-in plot to see (and demonstrate) the effect, as well as the full time series to see any negative consequences.

Problem 3: chain of spins

25 or 40 points

PHY410 students: do 3a and 3b only

PHY505 students: do full problem

This problem explores python speedup methods in the context of a 1D array of spins. This is related to the Ising model, which we will investigate in detail during the Monte Carlo part of the course. Given two spins with magnetic moments $\vec{m}_i$ and $\vec{m}_j$, with separation $\vec{r}_{ij}$, the potential energy of the dipole-dipole interaction is:

$$U_{ij} = \frac{\mu_0}{4\pi} \frac{\vec{m}_i \cdot \vec{m}_j - 3(\vec{m}_i \cdot \hat{r}_{ij})(\vec{m}_j \cdot \hat{r}_{ij})}{r_{ij}^3}.$$

and the total potential energy is:

$$U = \sum_{0 \leq i < j \leq N-1} U_{ij}.$$

Let's use a few simplifications:

1. The spins are placed on the x-axis, with a constant spacing a .
2. The spins point either up or down (say along the z axis): $\vec{m}_i = s_i m \hat{z}$, where $s_i = \pm 1$.
 - So, $\vec{m}_i \cdot \vec{m}_j = s_i s_j m^2$, and $\vec{m}_i \cdot \hat{r}_{ij} = 0$, since the spins are perpendicular to the spatial separation vector.
3. With appropriate choice of units, we can eliminate all of the constants from the equation (i.e., set $a = 1$ and $\epsilon = (\mu_0 m^2)/(4\pi a^3) = 1$).

With these simplifications, the potential energy is much simpler:

$$U = \sum_{0 \leq i < j \leq N-1} \frac{s_i s_j}{(|x_i - x_j|)^3}.$$

The framework for this problem has already been created in `problem3/problem3.ipynb` and accompanying files. Please do your work in this notebook.

Problem 3a

15 points

Implement the potential energy function in four ways:

1. A simple `for` loop in python;
2. A simple `for` loop in python, but accelerated using a numba decorator;
3. Using numpy arrays without any explicit `for` loops;
4. A C++ function using cppy (the structure is provided below, you just need to write the C++ function).

Demonstrate that all 4 functions return the same result on the test case, `s = np.array([+1, +1, -1, +1, +1])` (within floating point precision, that is).

Problem 3b

10 points

Following the example at `CompPhys/ReviewPython/bindings/example_pybind11_cpp.ipynb`, time each of the 4 methods as a function of input array size (the sizes to check are already written into the cell below). Make a matplotlib plot showing the time vs. array size, using a log-log plot. You can copy code directly from the example without attribution.

Problem 3c

15 points

- Using your results from (3b), perform a linear fit to $\log(t)$ as a function of $\log(N)$, where N is the array size and t is the time of execution. Restrict your fit to the linear region at larger array sizes, i.e., ignore the constant overhead that causes nonlinear behavior at low array sizes. You can use any fitting function (e.g. our function, numpy, etc.).
 - Print the fit results (parameters, uncertainties, and quality of fit). In your writeup, state whether the slope matches what you expect.
 - Use the linear fit to determine the maximum array size that your function can handle in 1 hour.
-